Factory Surveillance System

20-Step Technical Implementation Roadmap — End of Formation Project

MASTER PLAN

PHASE I: REQUIREMENT ANALYSIS & SYSTEM ARCHITECTURE

PHASE I — CORE PLANNING

01 **Define System Specifications & Use Cases**

Establish complete functional requirements. Identify primary personas (System Administrator, Zone Moderator, Auditor). Map out key capabilities: user login, automated session timeout, dynamic camera grid layout, PTZ controls, and real-time stream state handling.

Tools: UML Use Case Diagrams, Figma Mockups, Architectural Specifications.

02 **Design Relational Database Schema (RBAC Model)**

Construct an Entity-Relationship (ER) diagram for Role-Based Access Control. Create tables for Users, Roles, Zones, Cameras, and Role_Zone_Permissions. Ensure password hashes are allocated adequate character length for bcrypt outputs.

Tools: PostgreSQL / SQLite, ER Diagrams, Workbench.

PHASE II: ENVIRONMENT & VIDEO STREAM EMULATION

PHASE II — VIDEO PIPELINE PROOF-OF-CONCEPT

03 **Setup Simulated Factory RTSP Video Feeds**

Since live factory IP cameras are rarely available in development, create local RTSP servers. Configure MediaMTX (rtsp-simple-server) or use FFmpeg / OBS Studio to loop pre-recorded factory video files over RTSP endpoints.

Tools: MediaMTX, FFmpeg, OBS Studio, VLC Media Player.

04 **Configure RTSP to WebRTC/MSE Streaming Gateway**

Web browsers and Electron applications cannot play raw RTSP streams directly. Set up MediaMTX or go2rtc as a streaming proxy gateway. Test low-latency RTSP-to-WebRTC conversion to achieve stream delivery with low latency.

Tools: go2rtc, MediaMTX WebRTC engine, HLS.js fallback.

PHASE III: BACKEND API & SECURITY DEVELOPMENT

PHASE III — SECURE REST API INITIALIZATION

05 **Initialize Backend API Server & Database Connections**

Set up a clean server repository (Node.js/Express or Python/FastAPI). Implement database migrations, ORM connection pooling, and configure structured environment variable file handling for sensitive credentials.

Tools: Node.js (Express) or Python (FastAPI), Prisma / ORM, dotenv.

06 **Build Authentication & JWT Token Logic**

Implement POST /api/auth/login. Hash passwords using bcrypt with appropriate salt rounds. Generate signed JSON Web Tokens (JWT) upon successful credential verification, embedding user identity, assigned role ID, and permissions array.

Tools: JsonWebToken, Bcrypt, Postman API Tester.

PHASE III: BACKEND API & SECURITY (CONTINUED)

07 **Implement Permission Middleware & Camera Filtering**

Create server middleware that intercepts incoming REST calls, verifies the JWT header, and extracts user privileges. Build GET /api/cameras to return only the list of camera metadata and WebRTC URLs authorized for the user.

Tools: Express Middleware / FastAPI Dependencies, JWT Verification.

08 **Secure Stream Gateway Token Authorization**

Prevent unauthorized stream access via direct URL copying into external media players. Implement short-lived stream tokens or HTTP proxy tickets that validate against the Media Gateway before allowing video connection handshakes.

Tools: HMAC Stream Signing, WebSocket Auth Headers.

PHASE IV: FRONTEND APPLICATION DEVELOPMENT

PHASE IV — CLIENT UI & VIDEO GRID CONSTRUCTION

09 **Initialize React/Vue Frontend & State Management**

Bootstrap the frontend UI project using Vite with React or Vue 3. Set up styling and centralized state management (Zustand, Redux Toolkit, or Pinia) to handle moderator authentication state, stored JWT tokens, and active camera views.

Tools: React, Vite, CSS Modules, Axios, Zustand / Redux.

10 **Develop Secure Moderator Login Screen**

Build an intuitive, polished login interface. Include field validation, real-time error messaging for invalid credentials, loading states, and automatic redirection to the live surveillance dashboard upon successful authentication.

Tools: React Hook Form, Icons, Axios Interceptors.

11 **Create Dynamic Video Player Component**

Build a reusable React HTML5 Video player component using WebRTC native connection handlers or video.js. Program auto-reconnect routines to automatically attempt stream re-establishment if network connectivity drops.

Tools: HTML5 Video Element, WebRTC PeerConnection API, JS Video SDK.

12 **Build Multi-Camera Surveillance Grid Dashboard**

Construct the primary moderator window featuring layout controls (1x1 single view, 2x2 grid, 3x3 layout). Fetch permitted camera feeds from GET /api/cameras and render each stream dynamically inside the selected grid configuration.

Tools: CSS Grid, CSS Flexbox, DOM UI Controls.

13 **Implement Zone Selection & Single-Stream Modal**

Allow moderators to filter active feeds by factory zone (e.g., Assembly Line, Warehouse). Add a double-click feature on any grid video player to open a full-screen high-definition focus modal with overlay camera diagnostic data.

Tools: Fullscreen API, DOM Event Handlers.

PHASE V: DESKTOP INTEGRATION & ADMIN TOOLS

14 **Package Web Application into Electron Desktop App**

Wrap the completed React frontend into an Electron.js executable container. Configure native desktop window frame features: custom top titlebar, system tray minimization, hardware video acceleration flags, and auto-launch configuration.

Tools: Electron.js, Electron Builder, Node Integration Settings.

15 **Build Administrator CRUD Panel for Users & Cameras**

Create a dedicated management view accessible only to System Administrators. Allow admins to create new moderator accounts, assign role permissions, register new IP camera RTSP URLs, and update camera zone classifications dynamically.

Tools: React Admin Components, Data Tables, Modal Forms.

PHASE VI: TESTING, OPTIMIZATION & LOGGING

PHASE VI — SYSTEM HARDENING & QUALITY ASSURANCE

16 **Implement System Security Audit Logging**

Create an audit logging table in the database. Track and record critical security events: moderator login attempts, logout timestamps, permission violation attempts, and camera stream view duration for accountability.

Tools: Winston Logger, SQLite Audit Log Table, Middleware Logging.

17 **Perform Load & Video Performance Optimization**

Optimize browser memory utilization when running 9+ concurrent live streams. Tune hardware decoding flags, adjust bitrate streaming profiles in MediaMTX, and implement automatic canvas/video tearing prevention.

Tools: Chrome DevTools Performance Monitor, GPU Acceleration.

18 **Conduct End-to-End Security & RBAC Verification**

Perform integration testing. Log in as Moderator A (Zone A access only) and verify that API calls for Zone B camera feeds return HTTP 403 Forbidden. Confirm that expired JWT tokens automatically force redirection back to login.

Tools: Jest / Cypress, Postman Collection Test Runner.

PHASE VII: PACKAGING & ACADEMIC DEFENSE

PHASE VII — PROJECT FINALIZATION

19 **Build Executable Bundles & Containerization**

Compile the backend and Media Gateway into a Docker Compose file for easy deployment. Build the standalone Electron desktop client into a single executable binary installer (.exe for Windows or .AppImage for Linux).

Tools: Docker, Docker Compose, Electron Forge / Installer.

20 **Prepare Technical Written Report & Jury Defense Demo**

Write the final end-of-formation thesis report detailing system architecture, database ERD, security mechanisms, and user manual. Prepare a smooth live defense demonstration showcasing multi-user login with different permissions.

Tools: Presentation Slides, OBS Live Demo Setup, Thesis Document.